

CO5_ASSESSMENT 1

CONSTRAINT -BASED PROBLEM SOLVING

1. Real-Time Face Recognition for Campus Security

A university wants to deploy a camera-based security system at the entrance that can identify registered students and staff in real time.

The system should:

- Capture faces from live CCTV/video streams.
- Detect faces under different lighting conditions.
- Extract suitable facial features.
- Recognize registered individuals.
- Maintain real-time performance on a low-cost computer.
- Minimize false recognition.

Design a complete OpenCV-based face recognition pipeline from image acquisition to final identification. Explain the preprocessing, face detection, feature extraction, recognition, and optimization techniques used. Justify how your system achieves accuracy and real-time performance.

2. Intelligent Traffic Violation Detection System

A traffic department wants to automatically monitor vehicles using roadside cameras.

The system should:

- Detect vehicles from live video.
- Track vehicles across consecutive frames.
- Identify vehicles crossing a predefined line.
- Count vehicles accurately.
- Detect abnormal or unwanted movements.
- Display vehicle count and alerts in real time.

Design an OpenCV-based solution incorporating video acquisition, preprocessing, object detection, motion analysis, tracking, and visualization. Explain how you would handle shadows, camera noise, dynamic backgrounds, and multiple vehicles to minimize counting errors.

3. Automated Document Processing System

A company receives thousands of scanned documents that may contain blurred text, uneven illumination, noise, and different font sizes.

The system should:

- Automatically improve document quality.
- Remove noise and unwanted background.
- Convert the document into a suitable format for OCR.
- Segment text regions.
- Extract readable text efficiently.

- Process a large number of documents in batch mode.

Design an OpenCV-based image processing pipeline for this system. Explain the role of grayscale conversion, thresholding, filtering, morphological operations, segmentation, and geometric correction. Justify how each stage improves OCR accuracy.

4. Real-Time Multi-Person Tracking and Attendance System

A college wants to automatically record student attendance using classroom CCTV cameras.

The system should:

- Detect multiple faces simultaneously.
- Track each person across video frames.
- Recognize registered students.
- Avoid repeatedly marking the same student.
- Handle slight head movement and temporary face disappearance.
- Operate continuously with minimum processing delay.

Design a complete OpenCV-based system using face detection, feature extraction, recognition, and tracking. Explain how you maintain identity across frames and prevent false or duplicate attendance records.

5. Smart Home Surveillance and Intrusion Detection

A home security system uses a fixed camera to monitor an entrance continuously.

The system should:

- Detect significant motion.
- Distinguish actual movement from camera noise and small background changes.
- Track moving objects.
- Identify suspicious activity.
- Generate an alert when an unauthorized person enters a predefined region.
- Operate in real time with limited computational resources.

Design an OpenCV-based surveillance pipeline using background subtraction, filtering, morphological operations, contour/feature detection, motion analysis, and tracking. Explain how your approach reduces false alarms caused by shadows, illumination changes, trees, and other dynamic background objects.

ANSWERS

1. Real-Time Face Recognition for Campus Security

Introduction:

A real-time face recognition system can be developed using OpenCV to identify registered students and staff from live CCTV video. The system consists of **image acquisition, preprocessing, face detection, feature extraction, face recognition, decision-making, and optimization.**

System Pipeline:

CCTV Camera → Frame Acquisition → Preprocessing → Face Detection → Face Alignment → Feature Extraction → Face Matching → Identity Verification → Display/Alert

1. Image Acquisition

- Capture live video using an OpenCV camera interface.
- Each video frame is processed continuously.
- Resize frames to a smaller resolution to reduce computation.

2. Preprocessing

Preprocessing improves face detection under different conditions.

- Convert the frame to grayscale where appropriate.
- Apply histogram equalization or CLAHE to improve contrast.
- Use Gaussian filtering to reduce camera noise.
- Normalize face regions for consistent recognition.

3. Face Detection

OpenCV can use:

- Haar Cascade classifier for lightweight detection.
- DNN-based face detector when better robustness is required.

The detector identifies the coordinates of each face using bounding boxes.

4. Face Alignment

The detected face is cropped and aligned using facial landmarks such as the eyes and nose.

Alignment reduces variations caused by:

- Head tilt
- Small changes in face position
- Different camera angles

5. Feature Extraction

Important characteristics of the face are converted into numerical features.

A suitable approach is to use an OpenCV-compatible face embedding model. Each registered person has a stored feature vector.

6. Face Recognition

The extracted feature vector is compared with the registered database using:

- Euclidean distance, or
- Cosine similarity.

If the similarity is above a selected threshold, the person is identified. Otherwise, the system displays **Unknown**.

7. Minimizing False Recognition

To improve reliability:

- Use a strict recognition threshold.
- Require recognition across several consecutive frames.
- Maintain multiple reference images for each registered person.
- Reject low-quality or partially visible faces.
- Use an **Unknown** category instead of forcing a match.

8. Real-Time Optimization

For low-cost computers:

- Resize input frames.
- Process every second or third frame for recognition.
- Detect faces periodically and track them between detections.
- Use lightweight models.
- Use multithreading for video capture and processing.
- Avoid unnecessary image conversions.

Advantages:

- Automatic identification
- Continuous monitoring
- Reduced manual security work
- Fast recognition
- Can operate with relatively low-cost hardware

Conclusion:

The OpenCV pipeline combines preprocessing, face detection, feature extraction, recognition, and tracking. Proper thresholding, multiple-frame verification, and lightweight processing help achieve **high recognition accuracy while maintaining real-time performance.**

2. Intelligent Traffic Violation Detection System

Introduction:

An intelligent traffic monitoring system uses roadside cameras and OpenCV to detect, track, and count vehicles. It can also identify vehicles crossing restricted lines or entering prohibited regions.

System Pipeline:

Camera → Video Acquisition → Preprocessing → Vehicle Detection → Motion Analysis → Tracking → Line-Crossing Detection → Violation Detection → Counting → Alert/Visualization

1. Video Acquisition

- Capture continuous traffic video.
- Extract individual frames using OpenCV.
- Process frames at a suitable frame rate.

2. Preprocessing

Preprocessing improves the quality of traffic images.

- Resize frames.
- Apply Gaussian filtering to reduce noise.
- Improve contrast when lighting is poor.
- Convert to grayscale when required for motion analysis.

3. Vehicle Detection

Vehicles can be detected using:

- Background subtraction for fixed cameras.
- Contour detection for simple environments.
- A lightweight object detector for more reliable vehicle classification.

Detected vehicles are represented by bounding boxes.

4. Motion Analysis

For a fixed camera, background subtraction can identify moving regions.

Techniques include:

- MOG2
- KNN background subtraction
- Frame differencing

The foreground mask indicates regions containing moving objects.

5. Morphological Processing

Morphological operations clean the foreground mask.

Opening: removes small noise.

Closing: fills small gaps in detected objects.

Dilation: strengthens detected regions.

6. Vehicle Tracking

Each detected vehicle receives an ID.

A tracker follows the vehicle between consecutive frames. Possible tracking methods include:

- Centroid tracking
- Kalman filtering
- OpenCV tracking algorithms

This prevents the same vehicle from being counted repeatedly.

7. Line-Crossing Detection

A virtual line is drawn across the road.

When the center point of a tracked vehicle moves from one side of the line to the other:

Vehicle Count = Vehicle Count + 1

The vehicle ID is stored so that it cannot be counted again.

8. Abnormal Movement Detection

The system can identify:

- Wrong-direction movement
- Sudden stopping
- Unusual trajectories
- Entry into restricted regions

9. Handling Common Problems

Shadows:

Use color/brightness analysis and morphological filtering to reduce shadow regions.

Camera Noise:

Gaussian or median filtering reduces small random noise.

Dynamic Background:

Adaptive background models can update gradually when lighting or background conditions change.

Multiple Vehicles:

Use individual bounding boxes and unique tracking IDs.

Occlusion:

Tracking helps maintain the identity of vehicles temporarily hidden by another vehicle.

10. Visualization

The output screen displays:

- Bounding boxes
- Vehicle IDs
- Counting line
- Total vehicle count
- Violation messages
- Warning alerts

Conclusion:

The proposed system combines **OpenCV preprocessing, vehicle detection, motion analysis, tracking, and line-crossing detection.**

Proper filtering and tracking reduce duplicate counting and false alarms, allowing real-time traffic monitoring.

3. Automated Document Processing System

Introduction:

An automated document processing system improves scanned documents before sending them to an OCR engine. OpenCV can remove noise, correct illumination, enhance text, and segment text regions.

System Pipeline:

Scanned Document → Grayscale → Noise Removal → Illumination Correction → Thresholding → Morphological Processing → Text Segmentation → Geometric Correction → OCR → Text Output

1. Image Acquisition

Documents are obtained from:

- Scanners
- Cameras
- Digital images

Images are converted into a suitable format for processing.

2. Grayscale Conversion

A color document is converted to grayscale.

Benefits:

- Reduces computational complexity.
- Removes unnecessary color information.
- Makes text-background separation easier.

3. Noise Removal

Scanned documents may contain:

- Salt-and-pepper noise
- Scanner noise

- Small unwanted dots

Median or Gaussian filtering can reduce noise while preserving important text structures.

4. Illumination Correction

Uneven lighting can make some areas darker than others.

Techniques such as:

- Background estimation
- Adaptive thresholding
- Contrast enhancement

can improve text visibility.

5. Thresholding

Thresholding converts the grayscale image into a binary image.

Common techniques:

- Global thresholding
- Otsu thresholding
- Adaptive thresholding

Adaptive thresholding is especially useful when illumination is uneven.

6. Morphological Operations

Opening:

- Removes small unwanted objects.
- Reduces isolated noise.

Closing:

- Connects broken characters.
- Fills small gaps in text.

Dilation:

- Makes thin text regions stronger.

Erosion:

- Removes small unwanted regions and can refine characters.

7. Text Segmentation

The document is divided into meaningful regions such as:

- Paragraphs
- Lines
- Words
- Text blocks

Contours or connected-component analysis can be used to locate text regions.

8. Geometric Correction

Scanned pages may be:

- Rotated
- Skewed
- Perspective distorted

OpenCV can detect document boundaries and apply a perspective transformation.

This produces a properly aligned document for OCR.

9. OCR Preparation

After processing, the image should have:

- Clear characters
- Uniform background
- Correct orientation
- Appropriate contrast
- Reduced noise

The processed image is then passed to an OCR engine.

10. Batch Processing

For thousands of documents:

- Process files automatically from a folder.
- Use batch operations.
- Resize images when possible.
- Process independent documents in parallel.
- Save OCR output using corresponding filenames.

Conclusion:

The OpenCV pipeline improves OCR accuracy by systematically removing noise, correcting illumination, separating text from background, correcting document geometry, and segmenting text. Batch processing makes the system suitable for large-scale document processing.

4. Real-Time Multi-Person Tracking and Attendance System

Introduction:

A classroom attendance system can use CCTV video to detect multiple faces, recognize registered students, track their identities, and automatically record attendance without repeatedly marking the same student.

System Pipeline:

CCTV → Frame Capture → Preprocessing → Face Detection → Feature Extraction → Face Recognition → Multi-Person Tracking → Identity Association → Attendance Verification → Database

1. Video Acquisition

- Capture classroom video continuously.
- Resize frames for faster processing.
- Select suitable frames for face analysis.

2. Face Detection

An OpenCV face detector identifies multiple faces in each frame. Each detected face receives a bounding box.

3. Face Preprocessing

Detected faces are:

- Cropped
- Resized
- Normalized
- Aligned when possible

This makes feature extraction more consistent.

4. Feature Extraction

A face recognition model converts every detected face into a numerical feature representation.

Registered students have corresponding reference features stored in a database.

5. Face Recognition

The detected feature is compared with registered student features.

If the distance is below the recognition threshold:

Identity = Registered Student

Otherwise:

Identity = Unknown

6. Multi-Person Tracking

Each detected student is assigned a temporary tracking ID.

For example:

- Person 1 → Student A
- Person 2 → Student B
- Person 3 → Student C

The tracker follows their movement between frames.

7. Maintaining Identity

Identity can be maintained using:

- Face recognition
- Bounding-box position
- Centroid movement
- Appearance information
- Tracking IDs

If the face temporarily disappears, the tracker can maintain the person's ID for a limited number of frames.

8. Preventing Duplicate Attendance

A set or database table can store students who are already marked present.

Example:

Present = {Student A, Student B, Student C}

If Student A appears again, the system checks the record and does not create another attendance entry.

9. Multiple-Frame Verification

To reduce false recognition:

- Require recognition in several frames.
- Use confidence thresholds.
- Ignore very small or unclear faces.
- Confirm identity before recording attendance.

10. Handling Temporary Disappearance

If a student turns their head or is temporarily blocked:

- Tracking continues using the previous position.
- Recognition resumes when the face becomes visible.
- A short tracking timeout prevents immediate loss of identity.

11. Real-Time Optimization

- Resize frames.
- Detect faces periodically.
- Track between detections.
- Use lightweight models.
- Process multiple operations efficiently.
- Use separate capture and processing threads.

Conclusion:

The system combines **face detection, feature extraction, recognition, and multi-object tracking**. Maintaining tracking IDs and storing attendance records prevents duplicate entries, while multi-frame verification improves reliability.

5. Smart Home Surveillance and Intrusion Detection

Introduction:

A smart surveillance system can continuously monitor a fixed camera and detect significant movement. OpenCV can combine background subtraction, filtering, morphology, contours, and tracking to identify suspicious activity.

System Pipeline:

Camera → Frame Acquisition → Preprocessing → Background Subtraction → Noise Removal → Morphological Processing → Contour Detection → Object Tracking → ROI Analysis → Intrusion Decision → Alert

1. Video Acquisition

A fixed camera continuously captures the entrance area. OpenCV reads the video frame by frame.

2. Preprocessing

To reduce false detections:

- Resize frames.
- Apply Gaussian or median filtering.
- Convert to grayscale when suitable.
- Smooth small camera noise.

3. Background Subtraction

The system learns the normal background and separates moving objects from it.

Suitable OpenCV techniques include:

- MOG2

- KNN

The result is a foreground mask.

4. Morphological Processing

The foreground mask may contain small holes and noise.

Opening removes small noise.

Closing fills gaps in detected objects.

Dilation strengthens the detected foreground.

5. Contour Detection

Contours are extracted from the cleaned foreground mask.

Very small contours are ignored using an area threshold.

Large contours are considered potential moving objects.

6. Motion Analysis

The system calculates:

- Object position
- Movement direction
- Speed approximately from frame-to-frame displacement
- Time spent inside a region

This helps distinguish meaningful movement from small background changes.

7. Region of Interest

A predefined entrance region is created.

For example:

ROI = Door/Entrance Area

An alert is generated only when a tracked object enters this region.

8. Object Tracking

A detected object receives a tracking ID.

The system follows its position across consecutive frames.

This prevents the same person from generating multiple alerts.

9. Handling False Alarms

Camera Noise:

Use Gaussian/median filtering and minimum contour-area thresholds.

Shadows:

Use foreground-mask analysis and brightness/color information to distinguish shadows from solid moving objects.

Illumination Changes:

Use adaptive background models and gradual background updates.

Trees or Moving Objects:

Ignore small repetitive movements and use a region-of-interest restriction.

Small Background Changes:

Apply minimum-area and persistence thresholds.

10. Suspicious Activity Detection

An alert can be generated when:

- A person enters a restricted region.
- Movement continues for an unusual duration.
- An object moves in a prohibited direction.
- A tracked object remains inside the entrance area.

11. Real-Time Optimization

- Reduce frame resolution.
- Process only the required ROI.
- Use lightweight background subtraction.
- Track objects instead of detecting them in every frame.
- Avoid unnecessary image operations.

12. Visualization

The output displays:

- Motion regions
- Bounding boxes
- Tracking IDs
- ROI boundary
- Motion status
- Intrusion warning
- Alert timestamp

Conclusion:

The proposed OpenCV surveillance system combines **background subtraction, filtering, morphological operations, contour analysis, motion detection, and tracking**. Area thresholds, persistence checks, adaptive background models, and ROI-based detection significantly reduce false alarms while maintaining real-time operation.